CO1-AT2 APPLICATION BASED QUESTIONS

NAME: JEYA HARSHINI R

REG NO: 192424051

COURSE CODE: DSA0504

COURSE TITLE: QUERY PROCESSING FOR DATA SCIENCE

Title

Weather Monitoring System using Java, Python, XML and MySQL

1. Aim

To develop a Weather Monitoring System that reads weather information from an XML file, processes and cleans the data using Python, stores the processed data in a MySQL database, and displays it through a Java Swing desktop application.

2. Objective

- To read weather data from an XML file.
- To clean the dataset by removing duplicate records and filling missing values.
- To store the processed data in a MySQL database.
- To retrieve weather records using JDBC.
- To display the processed weather information in a Java Swing GUI.

3. Description of Algorithm

Algorithm

1. Start the application.

2. Read weather data from the XML file.
3. Convert XML data into a DataFrame using Python.
4. Remove duplicate weather records.
5. Replace missing temperature values with the average temperature.
6. Generate a status ("Hot", "Warm", or "Cool") based on the temperature.
7. Save the cleaned data into a CSV file.
8. Insert the cleaned records into the MySQL database.
9. Connect the Java application to MySQL using JDBC.
10. Retrieve all weather records from the database.
11. Display the records in a JTable.
12. Stop the application.

---

4. Input

Screenshot 1 (Input XML)

Figure 1: Input Weather XML File

---

5. Output Screenshot of Application

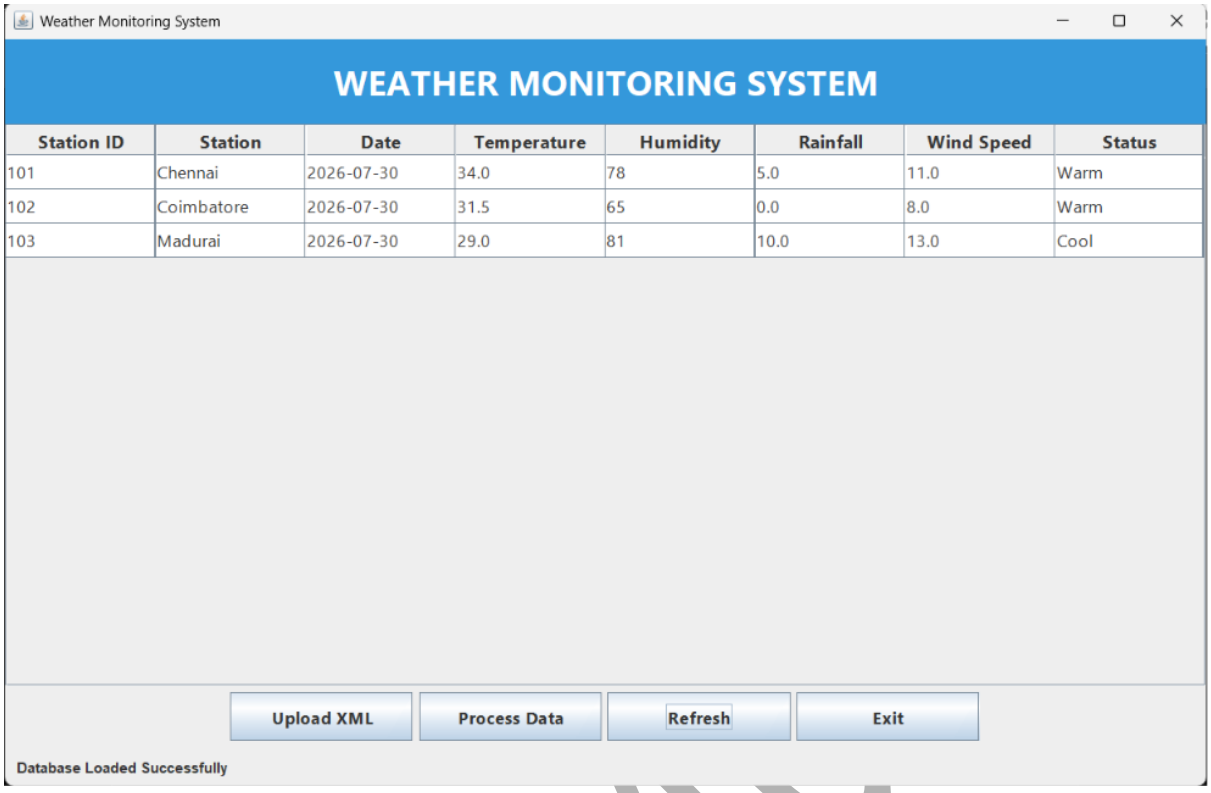


Figure 2: Final Weather Monitoring System Output

---

6. Code Screenshot

Screenshot 1

Main.java

Figure 3: Main Method

Screenshot 2

Dashboard.java

```
frontend > Dashboard.java
 9  public class Dashboard extends JFrame {
23      public Dashboard() {
51          model.addColumn("Date");
52          model.addColumn("Temperature");
53          model.addColumn("Humidity");
54          model.addColumn("Rainfall");
55          model.addColumn("Wind Speed");
56          model.addColumn("Status");
57
58          table = new JTable(model);
59
60          table.setRowHeight(28);
61          table.setFont(new Font("Segoe UI", Font.PLAIN, 14));
62          table.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 15));
63
64          JScrollPane scrollPane = new JScrollPane(table);
65          add(scrollPane, BorderLayout.CENTER);
66
67          // ---------------- BOTTOM PANEL ----------------
68
69          JPanel bottomPanel = new JPanel(new BorderLayout());
70
71          JPanel buttonPanel = new JPanel();
72
73          uploadButton = new JButton("Upload XML");
74          processButton = new JButton("Process Data");
75          refreshButton = new JButton("Refresh");
76          exitButton = new JButton("Exit");
77
```

Figure 4: Java Swing Dashboard

Screenshot 3

Dashboard.java

```
processButton.addActionListener(e -> {

    try {

        ProcessBuilder pb = new ProcessBuilder(
                "python",
                "backend/process_weather.py"
        );

        pb.inheritIO();
```

Figure 5: Process Button Event

Screenshot 4

Dashboard.java

```java
private void loadWeatherData() {

    try {

        model.setRowCount(0);

        Connection con = DBConnection.getConnection();

        Statement stmt = con.createStatement();

        ResultSet rs = stmt.executeQuery("SELECT * FROM weather");

        while (rs.next()) {

            model.addRow(new Object[]{

                    rs.getInt("station_id"),
                    rs.getString("station_name"),
                    rs.getDate("weather_date"),
                    rs.getDouble("temperature"),
                    rs.getInt("humidity"),
                    rs.getDouble("rainfall"),
                    rs.getDouble("wind_speed"),
                    rs.getString("status")
```

Figure 6: Loading Data from MySQL

Screenshot 5

frontend > DBConnection.java

```java
import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnection {

    private static final String URL = "jdbc:mysql://localhost:3306/weather_db";
    private static final String USER = "root";
    private static final String PASSWORD = "HarDar@_1872";

    public static Connection getConnection() throws Exception {

        return DriverManager.getConnection(
                URL,
                USER,
                PASSWORD
        );

    }
```

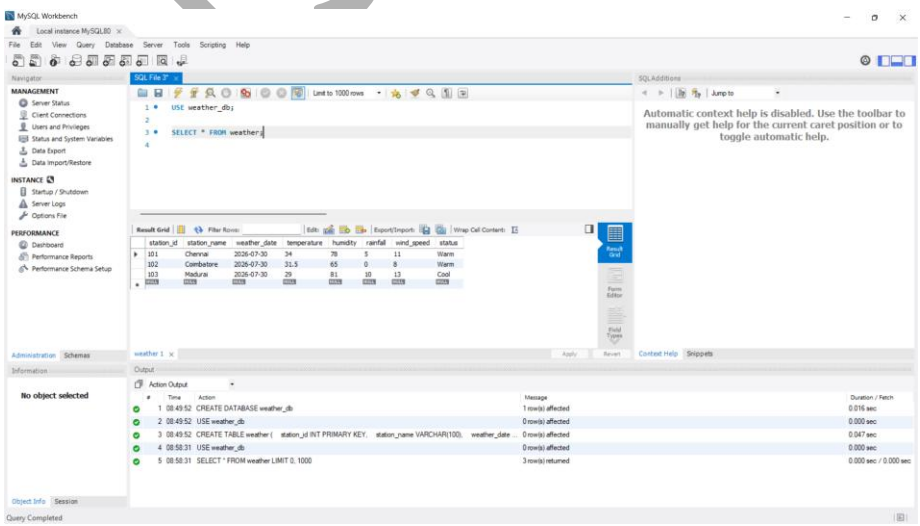
Figure 7: Database Connection

Screenshot 6

process_weather.py

```
# ----------------------------
# Insert into MySQL
# ----------------------------
connection = get_connection()
cursor = connection.cursor()

# Optional: clear existing data before inserting
cursor.execute("DELETE FROM weather")

for _, row in df.iterrows():
    cursor.execute("""
        INSERT INTO weather
        (station_id, station_name, weather_date, temperature,
         humidity, rainfall, wind_speed, status)
        VALUES (%s,%s,%s,%s,%s,%s,%s,%s)
    """, (
        int(row["station_id"]),
        row["station_name"],
        row["weather_date"],
        float(row["temperature"]),
        int(row["humidity"]),
        float(row["rainfall"]),
        float(row["wind_speed"]),
        row["status"]
    ))

connection.commit()

print("\n✓ Data inserted into MySQL successfully!")
```

Figure 8: Python Data Processing/INSERTION



7. Conclusion

The Weather Monitoring System was successfully developed using Java Swing, Python, XML, and MySQL. The application reads weather data from an XML file, performs data cleaning, stores the processed records in the database, and displays them through a graphical interface. This project demonstrates the integration of frontend, backend, and database technologies in a desktop application.

Title

Retail Sales Data Transformation and Integration System Using Python

---

Aim

To develop a Python-based retail sales data transformation system that imports retail datasets from machine-readable formats, performs data wrangling and integration, removes inconsistent and duplicate records, and generates a consolidated dataset suitable for loading into a relational database for business reporting.

Objective

- To import retail sales data from machine-readable datasets.
- To clean and preprocess retail data by handling missing and duplicate values.
- To transform the datasets into a consistent format.
- To integrate retail information using common fields.
- To generate a consolidated dataset for further business analysis and reporting.

Description of Algorithm

Algorithm

1. Start the program.
2. Import the retail sales dataset (CSV format).
3. Load product information and supplier information (if available) into DataFrames.

4. Perform data wrangling by:
    - Removing duplicate records.
    - Handling missing values.
    - Standardizing column names and data types.
5. Integrate the datasets using common fields such as Product ID or Category.
6. Perform feature engineering to prepare the data for analysis.
7. Save the cleaned and integrated dataset.
8. Use the transformed dataset for dashboard visualization and business reporting.
9. Stop the program.

Input

Input Dataset

The application uses the **Sample Superstore Dataset** containing retail transaction details such as:

- Order ID
- Product Name
- Category
- Sales
- Profit
- Quantity
- Region
- Customer Segment
- Shipping Mode

```
Row ID,Order ID,Order Date,Ship Date,Ship Mode,Customer ID,Customer Name,Segment,Country,Cit
1,CA-2016-152156,11/8/2016,11/11/2016,Second Class,CG-12520,Claire Gute,Consumer,United Stat
2,CA-2016-152156,11/8/2016,11/11/2016,Second Class,CG-12520,Claire Gute,Consumer,United Stat
3,CA-2016-138688,6/12/2016,6/16/2016,Second Class,DV-13045,Darrin Van Huff,Corporate,United
4,US-2015-108966,10/11/2015,10/18/2015,Standard Class,SO-20335,Sean O'Donnell,Consumer,Unite
5,US-2015-108966,10/11/2015,10/18/2015,Standard Class,SO-20335,Sean O'Donnell,Consumer,Unite
6,CA-2014-115812,6/9/2014,6/14/2014,Standard Class,BH-11710,Brosina Hoffman,Consumer,United
```

Figure 1: Sample Superstore Dataset (CSV Input)

Output Screenshot of Application

The processed retail dataset is visualized through the SmartRetail dashboard, showing business insights and analytical results after preprocessing and transformation.

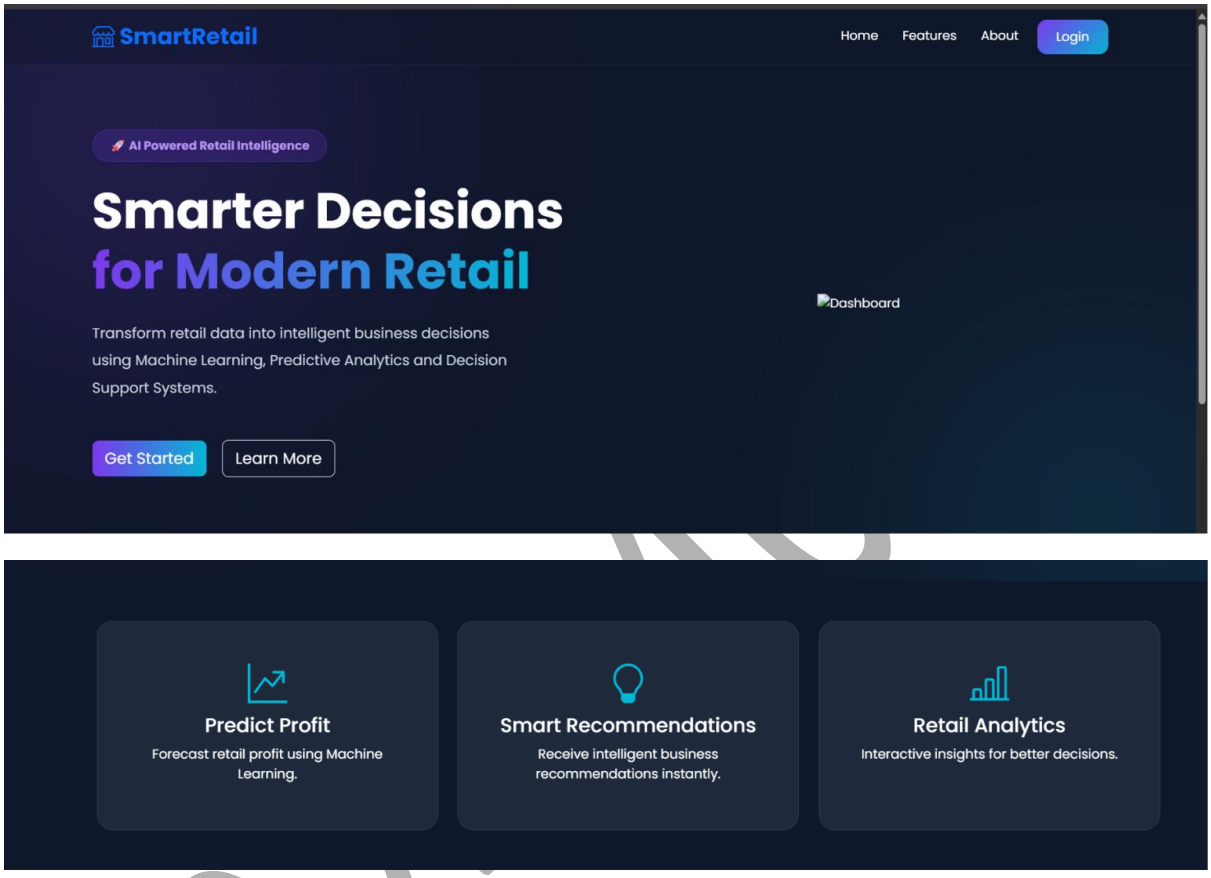


Figure 2: SmartRetail Dashboard

Code Screenshots

1. Data Preprocessing (preprocessing.py)

```python
# Convert date columns to datetime format
df["Order Date"] = pd.to_datetime(df["Order Date"])
df["Ship Date"] = pd.to_datetime(df["Ship Date"])

print("\nUpdated Data Types:")
print(df.dtypes)
# Extract date features
df["Order Year"] = df["Order Date"].dt.year
df["Order Month"] = df["Order Date"].dt.month_name()
df["Order Quarter"] = df["Order Date"].dt.quarter

# Calculate shipping duration
df["Shipping Days"] = (
    df["Ship Date"] - df["Order Date"]
).dt.days
```

Figure 3: Data Cleaning and Preprocessing

---

2. Feature Engineering (feature_engineering.py)

```python
# -------------------------
# Shipping Duration
# -------------------------
df["Shipping Days"] = (
    df["Ship Date"] - df["Order Date"]
).dt.days

# -------------------------
# Profit Margin
# -------------------------
df["Profit Margin"] = (
    (df["Profit"] / df["Sales"]) * 100
).round(2)
```

Figure 4: Feature Engineering

---

3. Exploratory Data Analysis (eda.py)

```python
# Convert dates
df["Order Date"] = pd.to_datetime(df["Order Date"])
df["Ship Date"] = pd.to_datetime(df["Ship Date"])

# Add new features
df = add_features(df)
sales_category = (
    df.groupby("Category")["Sales"]
    .sum()
    .sort_values(ascending=False)
)
```

Figure 5: Exploratory Data Analysis

---

4. Prediction Module (prediction.py)

```python
# ----------------------------
# Random Forest Regressor
# ----------------------------

random_forest = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)
random_forest.fit(X_train, y_train)

print("\nRandom Forest Model Trained Successfully!")
rf_predictions = random_forest.predict(X_test)
rf_mae = mean_absolute_error(y_test, rf_predictions)
rf_mse = mean_squared_error(y_test, rf_predictions)
rf_rmse = np.sqrt(rf_mse)
rf_r2 = r2_score(y_test, rf_predictions)

print("\n========== Random Forest ==========")
print("Mean Absolute Error :", rf_mae)
print("Mean Squared Error  :", rf_mse)
print("Root Mean Squared Error :", rf_rmse)
print("R² Score :", rf_r2)
```

Figure 6: Prediction Module

---

5. Recommendation Module (recommendation.py)

```python
def generate_recommendation(predicted_profit):

    if predicted_profit < 0:
        return (
            "Loss Expected",
            "Reduce discounts, review pricing strategy, and optimize inventory."
        )

    elif predicted_profit < 100:
        return (
            "Low Profit",
            "Increase promotional activities and improve product visibility."
        )

    elif predicted_profit < 500:
        return (
            "Moderate Profit",
            "Maintain the current strategy while monitoring sales trends."
        )

    else:
        return (
            "High Profit",
            "Increase inventory and marketing efforts to maximize revenue."
```

Figure 7: Recommendation Module

---

6. Main Application (app.py)

app.py > ...

```python
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def home():
    return render_template("index.html")

if __name__ == "__main__":
    app.run(debug=True)
```

Figure 8: Main Application

Conclusion

The Retail Sales Data Transformation System successfully imports and preprocesses retail sales data, performs data wrangling by removing inconsistencies and duplicate records, and generates a clean dataset suitable for business analysis. The transformed data is further utilized in the SmartRetail dashboard to support visualization, prediction, and informed decision-making.

For your **CO1 – AT2 Assessment**, keep it concise and aligned with what you actually built.

CO1 – AT2 Assessment

Project Title

The Book Nook – Library Database Preparation System

Aim

To develop a Flask-based Library Database Preparation System that reads book details from a JSON file, validates the records, generates a cleaned dataset, stores valid records in a MySQL database, and displays the library collection through a web interface.

Objective

- To read book records from a JSON file.
- To validate mandatory fields such as Book ID, Title, Author, Publisher, and Year.
- To detect missing or invalid records.
- To generate a cleaned JSON dataset containing only valid records.
- To insert validated records into the MySQL database.
- To display the stored books using a Flask web application.

Description of Algorithm

- Start the application.
- Connect to the MySQL database.
- Display the Home page of the Book Nook application.
- Navigate to the Add Book page.
- Upload the library JSON file.
- Read and parse the uploaded JSON data.
- Validate each book record by checking the required fields:

    - Book ID
    - Title
    - Author
    - Publisher
    - Year
- Identify and reject records with missing fields, invalid publication year, or duplicate Book IDs.
- Store all valid records in a cleaned dataset (cleaned_books.json).
- Insert the validated records into the MySQL database.
- Retrieve all stored book records from the database.
- Display the validated books on the Library Collection page.
- End the application.

Input

The input is a **JSON file** containing library book records.

Example:

books.json > ...

```json
[
    {
        "book_id": 101,
        "title": "Harry Potter",
        "author": "J.K. Rowling",
        "publisher": "Bloomsbury",
        "year": 1997
    },
    {
        "book_id": 102,
        "title": "",
        "author": "Dan Brown",
        "publisher": "Penguin",
        "year": 2003
    },
```

Output

The application performs the following operations:

- Reads the uploaded JSON file.
- Validates all book records.
- Removes invalid or duplicate records.
- Generates a cleaned JSON dataset (cleaned_books.json).
- Inserts only valid records into the MySQL database.
- Displays the validated books in the Book Nook web application.

Insert the following screenshots:

1. Home Page
2. Upload JSON Page
3. Library Collection Page
4. MySQL Database (books table)
5. cleaned_books.json
6. Terminal Output showing validation results
7. Code screenshots (app.py, index.html, books.html)

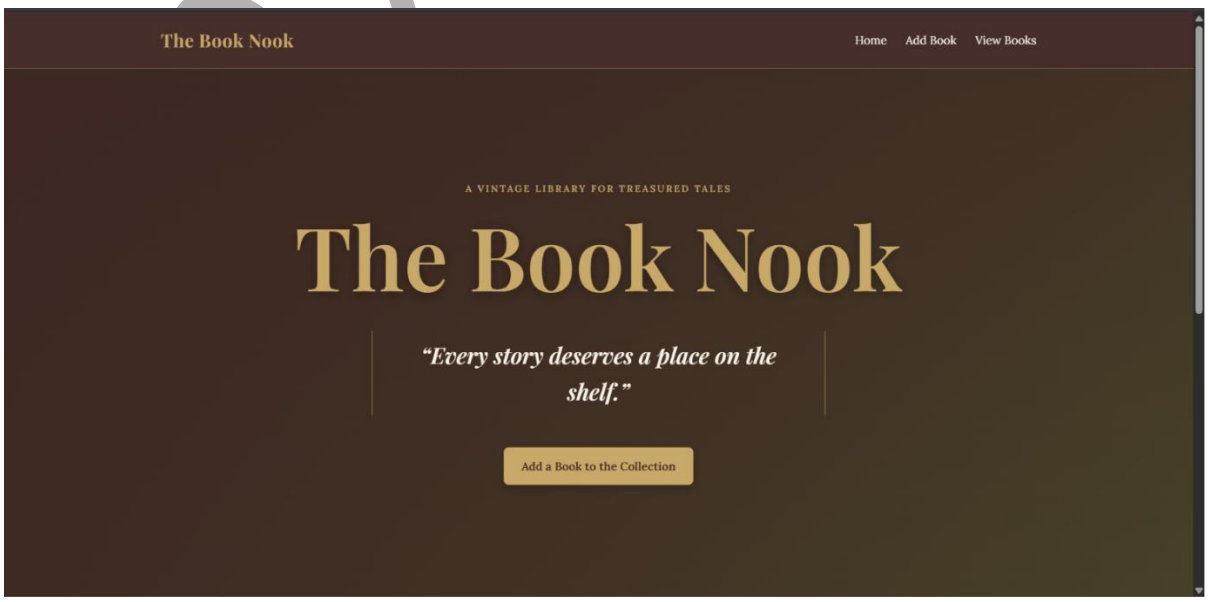


Figure 1: Home Page

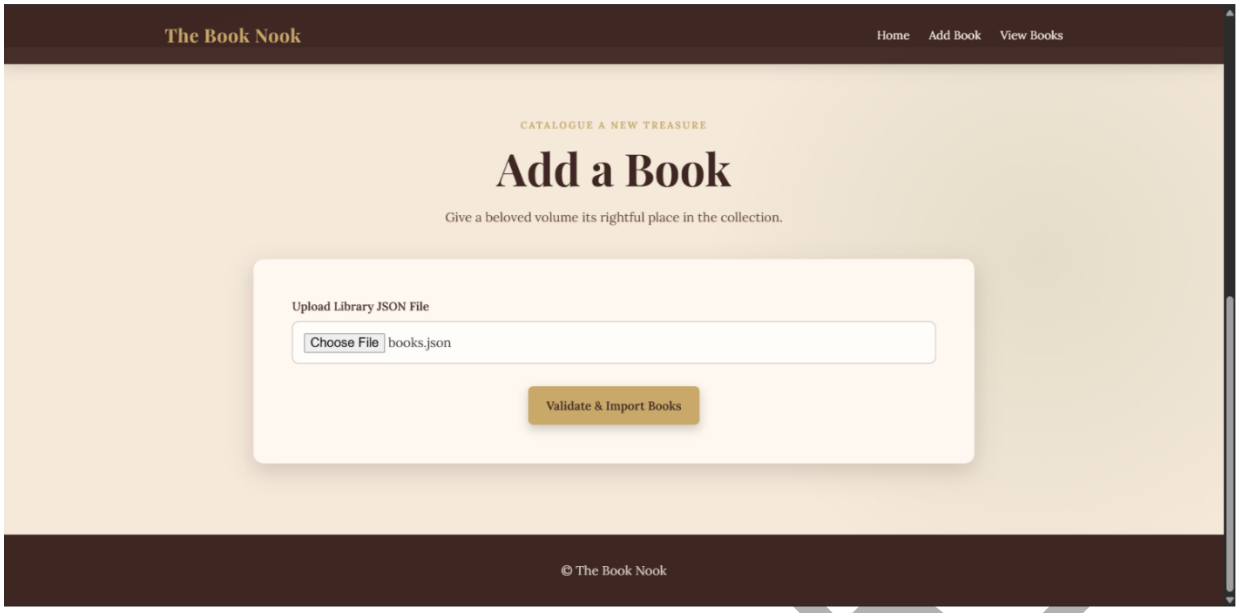


Figure 2: Upload JSON File

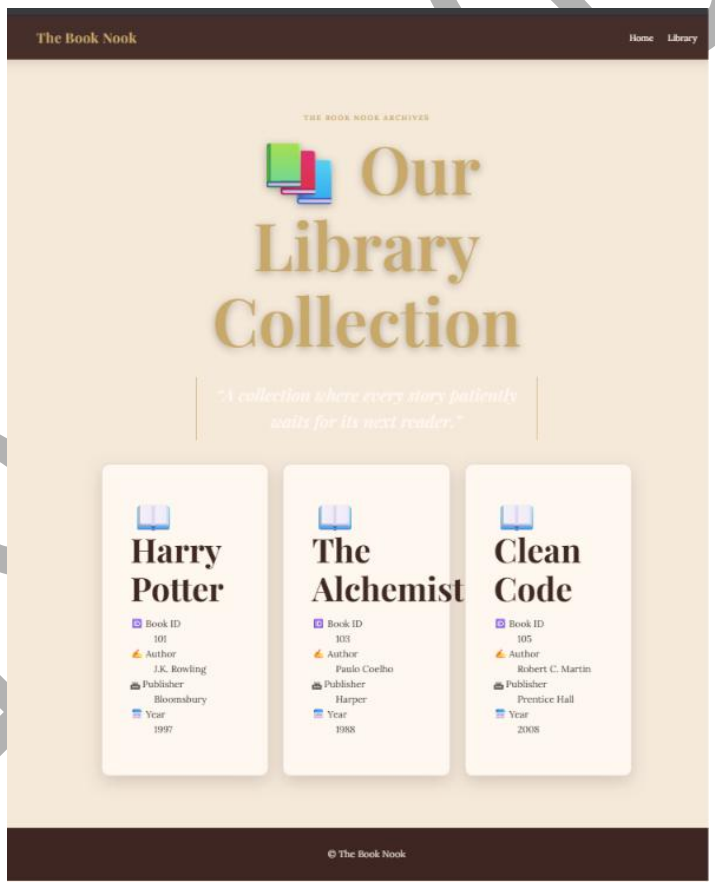


Figure 3: Library Collection

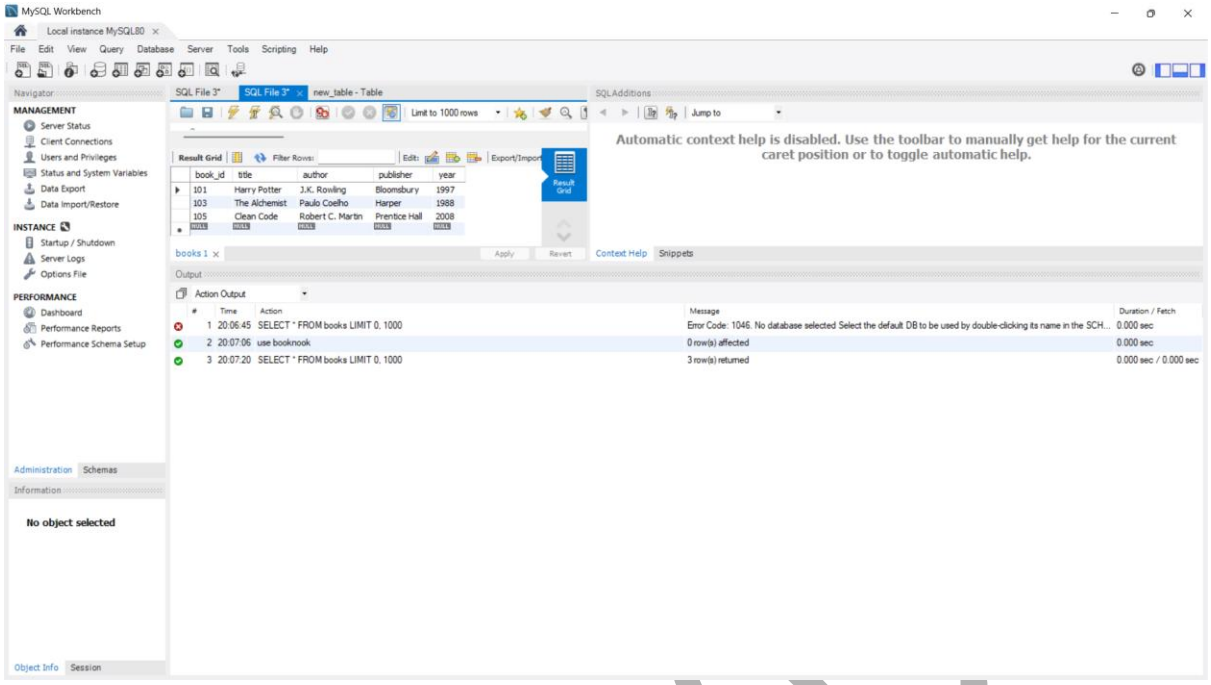


Figure 4: MySQL Database

Figure 5: Cleaned JSON Dataset

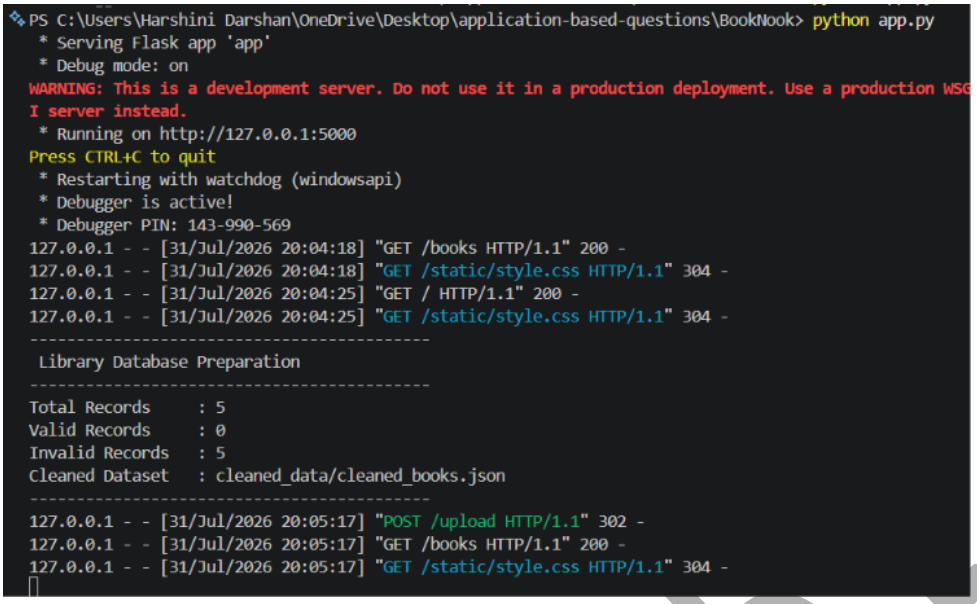
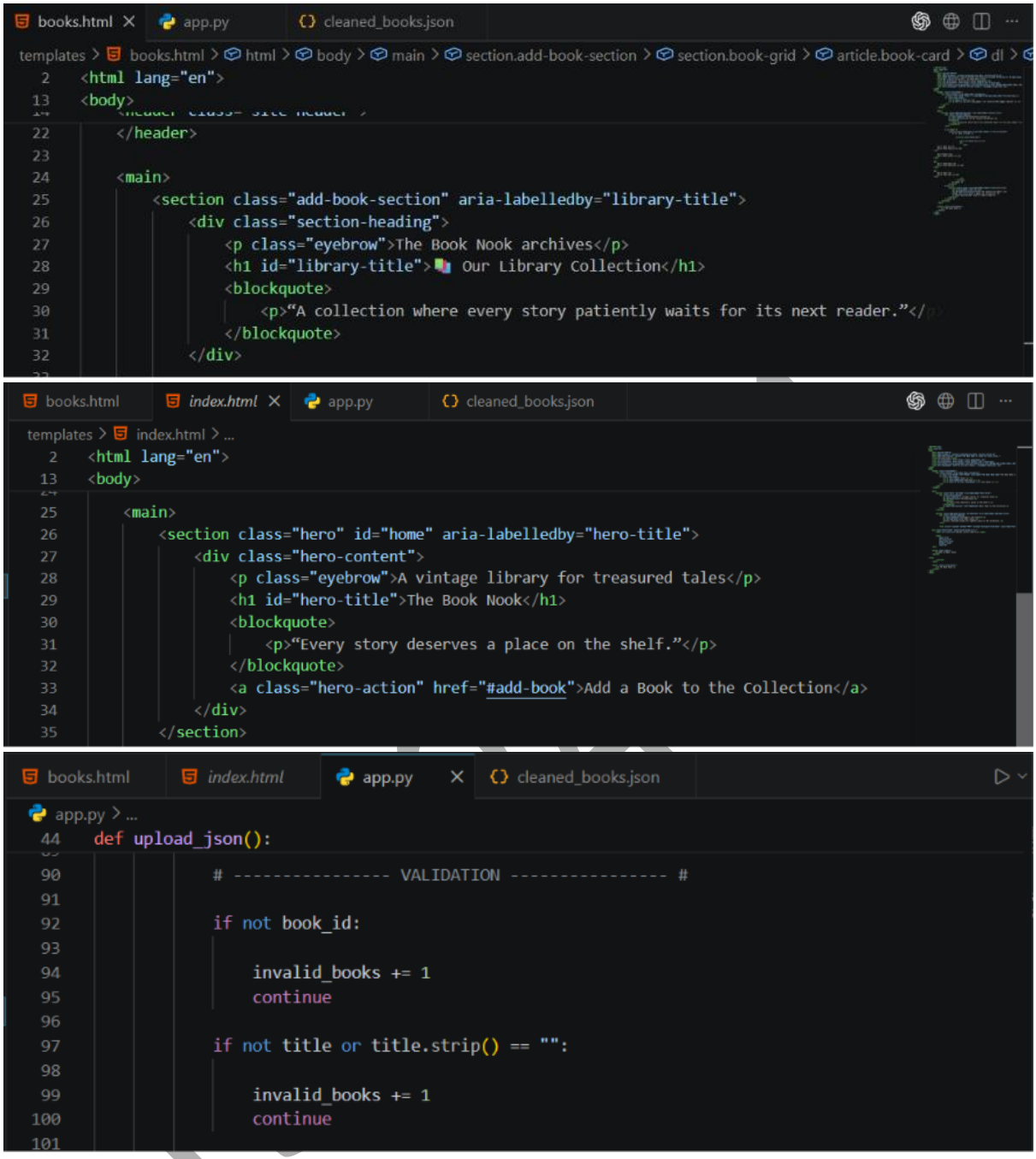


Figure 6: Terminal Output



```
books.html | app.py | cleaned_books.json
templates > books.html > html > body > main > section.add-book-section > section.book-grid > article.book-card > dl
 2  <html lang="en">
13  <body>
22      </header>
23
24      <main>
25          <section class="add-book-section" aria-labelledby="library-title">
26              <div class="section-heading">
27                  <p class="eyebrow">The Book Nook archives</p>
28                  <h1 id="library-title">📚 Our Library Collection</h1>
29                  <blockquote>
30                      <p>“A collection where every story patiently waits for its next reader.”</
31                  </blockquote>
32              </div>
```

```
books.html | index.html | app.py | cleaned_books.json
templates > index.html > ...
 2  <html lang="en">
13  <body>
25      <main>
26          <section class="hero" id="home" aria-labelledby="hero-title">
27              <div class="hero-content">
28                  <p class="eyebrow">A vintage library for treasured tales</p>
29                  <h1 id="hero-title">The Book Nook</h1>
30                  <blockquote>
31                      <p>“Every story deserves a place on the shelf.”</p>
32                  </blockquote>
33                  <a class="hero-action" href="#add-book">Add a Book to the Collection</a>
34              </div>
35          </section>
```

```
books.html | index.html | app.py | cleaned_books.json
app.py > ...
44  def upload_json():
90              # ---------------- VALIDATION ---------------- #
91
92              if not book_id:
93
94                  invalid_books += 1
95                  continue
96
97              if not title or title.strip() == "":
98
99                  invalid_books += 1
100                 continue
101
```

Figure 7: Code (app.py, index.html, books.html)

Conclusion

The **Book Nook – Library Database Preparation System** successfully processes library book records received in JSON format. It validates the required fields, detects invalid and duplicate records, generates a cleaned JSON dataset, stores only valid records in the MySQL database, and displays the validated library collection through a Flask-based web interface. The project demonstrates practical implementation of JSON

processing, data validation, data cleaning, database integration, and web application development using Python, Flask, HTML, CSS, and MySQL.

Project Title

Employee Payroll Preparation System

Aim

To develop an Employee Payroll Preparation System using Python and Flask that processes employee and salary data from CSV and JSON files, calculates payroll details, generates reports, and displays the payroll information through a web interface.

Objective

- To read employee details from a CSV file.
- To read salary details from a JSON file.
- To merge employee and salary datasets.
- To identify employees with missing salary records.
- To calculate the total salary of each employee.
- To classify employees based on salary grades.
- To generate payroll reports and cleaned payroll datasets.
- To display payroll details using a Flask web application.

Description of Algorithm

1. Start the Employee Payroll Preparation System.
2. Read employee details from the **employees.csv** file.
3. Read salary details from the **salary.json** file.
4. Merge employee and salary data using the Employee ID.
5. Check for employees with missing salary records.
6. Replace missing salary values with zero.
7. Calculate the total salary by adding Basic Salary, HRA, and Allowance.

8. Assign a salary grade (A, B, C, or D) based on the total salary.
9. Generate the **payroll_report.csv** file containing all employee payroll details.
10. Generate the **cleaned_payroll.csv** file containing only valid salary records.
11. Display payroll statistics and employee details on the Flask web application.
12. Stop the application.

Input

The application uses two input files:

Employee Dataset (CSV)

Contains:

- Employee ID
- Employee Name
- Department ID
- Designation

Salary Dataset (JSON)

Contains:

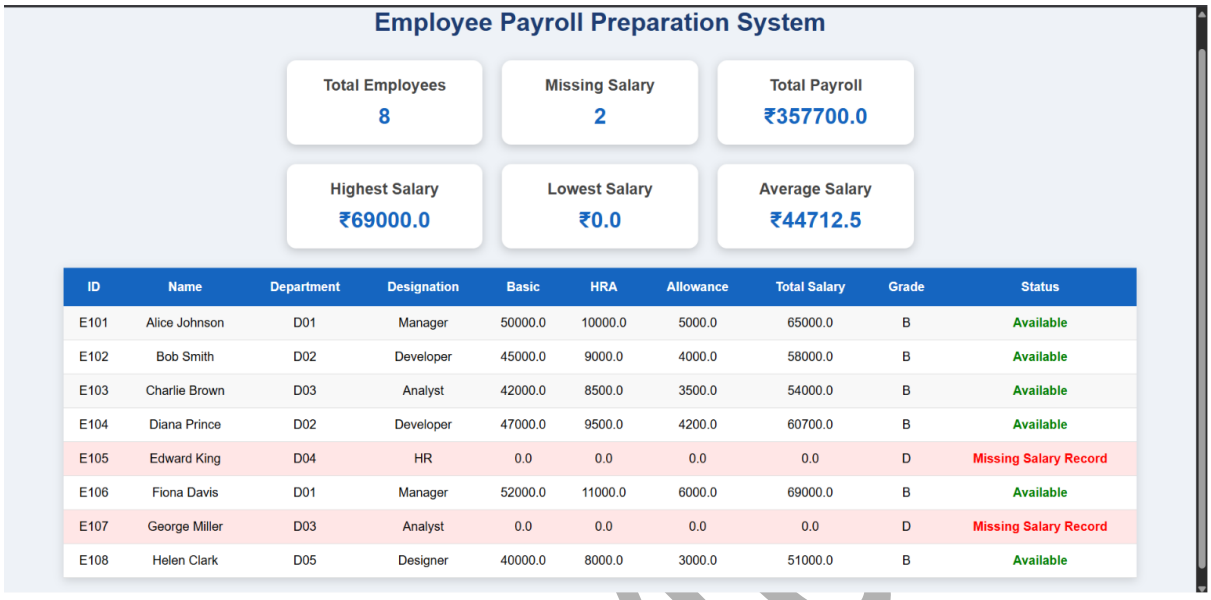
- Employee ID
- Basic Salary
- HRA
- Allowance

Output

The application successfully:

- Loads employee and salary datasets.
- Merges employee and salary information.
- Detects missing salary records.
- Calculates the total salary of each employee.
- Assigns salary grades.

- Generates payroll reports.
- Creates a cleaned payroll dataset.
- Displays payroll statistics and employee details through the Flask web interface.



ID	Name	Department	Designation	Basic	HRA	Allowance	Total Salary	Grade	Status
E101	Alice Johnson	D01	Manager	50000.0	10000.0	5000.0	65000.0	B	Available
E102	Bob Smith	D02	Developer	45000.0	9000.0	4000.0	58000.0	B	Available
E103	Charlie Brown	D03	Analyst	42000.0	8500.0	3500.0	54000.0	B	Available
E104	Diana Prince	D02	Developer	47000.0	9500.0	4200.0	60700.0	B	Available
E105	Edward King	D04	HR	0.0	0.0	0.0	0.0	D	Missing Salary Record
E106	Fiona Davis	D01	Manager	52000.0	11000.0	6000.0	69000.0	B	Available
E107	George Miller	D03	Analyst	0.0	0.0	0.0	0.0	D	Missing Salary Record
E108	Helen Clark	D05	Designer	40000.0	8000.0	3000.0	51000.0	B	Available

Figure 1: Employee Payroll Dashboard

```
PS C:\Users\Harshini Darshan\OneDrive\Desktop\application-based-questions\EmployeePayrollSystem> pyth
on app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSG
I server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with watchdog (windowsapi)
 * Debugger is active!
 * Debugger PIN: 143-990-569

==========================================
      EMPLOYEE PAYROLL PREPARATION
==========================================
Total Employees        : 8
Valid Salary Records   : 6
Missing Salary Records : 2
------------------------------------------
Total Payroll          : ₹357,700.00
Highest Salary         : ₹69,000.00
Lowest Salary          : ₹0.00
Average Salary         : ₹44,712.50
------------------------------------------
Payroll Report Saved   : data/payroll_report.csv
Cleaned Dataset Saved  : data/cleaned_payroll.csv
==========================================

127.0.0.1 - - [31/Jul/2026 20:46:11] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [31/Jul/2026 20:46:11] "GET /static/style.css HTTP/1.1" 200 -
```

Figure 2: Terminal Output

data > employees.csv

```
employee_id,name,department_id,designation
E101,Alice Johnson,D01,Manager
E102,Bob Smith,D02,Developer
E103,Charlie Brown,D03,Analyst
E104,Diana Prince,D02,Developer
E105,Edward King,D04,HR
E106,Fiona Davis,D01,Manager
E107,George Miller,D03,Analyst
E108,Helen Clark,D05,Designer
```

Figure 3: Employee Dataset (employees.csv)

data > salary.json > ...

```
[
    {
        "employee_id": "E101",
        "basic": 50000,
        "hra": 10000,
        "allowance": 5000
    },
    {
        "employee_id": "E102",
        "basic": 45000,
        "hra": 9000,
        "allowance": 4000
    },
    {
```

Figure 4: Salary Dataset (salary.json)

data > payroll_report.csv

```
employee_id,name,department_id,designation,basic,hra,allowance,Salary Status,to
E101,Alice Johnson,D01,Manager,50000.0,10000.0,5000.0,Available,65000.0,B
E102,Bob Smith,D02,Developer,45000.0,9000.0,4000.0,Available,58000.0,B
E103,Charlie Brown,D03,Analyst,42000.0,8500.0,3500.0,Available,54000.0,B
E104,Diana Prince,D02,Developer,47000.0,9500.0,4200.0,Available,60700.0,B
E105,Edward King,D04,HR,0.0,0.0,0.0,Missing Salary Record,0.0,D
E106,Fiona Davis,D01,Manager,52000.0,11000.0,6000.0,Available,69000.0,B
E107,George Miller,D03,Analyst,0.0,0.0,0.0,Missing Salary Record,0.0,D
E108,Helen Clark,D05,Designer,40000.0,8000.0,3000.0,Available,51000.0,B
```

Figure 5: Generated Payroll Report (payroll_report.csv)

data > cleaned_payroll.csv

```
employee_id,name,department_id,designation,basic,hra,allowance,Salary Status,tota
E101,Alice Johnson,D01,Manager,50000.0,10000.0,5000.0,Available,65000.0,B
E102,Bob Smith,D02,Developer,45000.0,9000.0,4000.0,Available,58000.0,B
E103,Charlie Brown,D03,Analyst,42000.0,8500.0,3500.0,Available,54000.0,B
E104,Diana Prince,D02,Developer,47000.0,9500.0,4200.0,Available,60700.0,B
E106,Fiona Davis,D01,Manager,52000.0,11000.0,6000.0,Available,69000.0,B
E108,Helen Clark,D05,Designer,40000.0,8000.0,3000.0,Available,51000.0,B
```

Figure 6: Cleaned Payroll Dataset (cleaned_payroll.csv)

app.py > ...

```python
def home():
    # ---------- Generate Payroll ---------- #

    payroll = generate_payroll(
        employees,
        salary
    )

    # ---------- Save Reports ---------- #

    save_report(payroll)

    # ---------- Dashboard Statistics ---------- #

    total_employees = len(payroll)

    missing_salary = len(
        payroll[
            payroll["Salary Status"] == "Missing Salary Record"
        ]
```

src > report_generator.py > print_summary

```python
def save_report(payroll_df):

    # ---------- Console Output ---------- #

    print("\n==========================================")
    print("      EMPLOYEE PAYROLL PREPARATION")
    print("==========================================")

    print(f"Total Employees        : {total_employees}")
    print(f"Valid Salary Records   : {valid_records}")
    print(f"Missing Salary Records : {missing_records}")

    print("------------------------------------------")

    print(f"Total Payroll          : ₹{total_payroll:,.2f}")
    print(f"Highest Salary         : ₹{highest_salary:,.2f}")
    print(f"Lowest Salary          : ₹{lowest_salary:,.2f}")
    print(f"Average Salary         : ₹{average_salary:,.2f}")

    print("------------------------------------------")

    print("Payroll Report Saved   : data/payroll_report.csv")
    print("Cleaned Dataset Saved  : data/cleaned_payroll.csv")

    print("==========================================\n")
```

Figure 7: Source Code (app.py, payroll.py, report_generator.py)

Conclusion

The **Employee Payroll Preparation System** successfully automates the payroll preparation process by integrating employee information from CSV files and salary information from JSON files. The application calculates total salary, identifies employees with missing salary records, assigns salary grades, and generates payroll reports along with a cleaned payroll dataset. The Flask-based web interface provides a simple and organized view of payroll statistics and employee information. The project demonstrates practical implementation of file handling, data processing, report generation, and web application development using Python, Pandas, Flask, HTML, CSS, CSV, and JSON.

Project Title

MobileMart Analytics – Mobile Phone Sales Analysis System

Aim

To develop a Mobile Phone Sales Analysis System using Python and Flask that reads product details from a JSON file and sales transactions from a CSV file, combines the data, analyzes sales performance, and generates a consolidated sales report.

Objective

- To read product information from a JSON file.
- To read sales transaction details from a CSV file.
- To merge both datasets using Product ID.
- To calculate the total units sold and revenue for each product.
- To identify the best-selling mobile phone.
- To generate a consolidated sales report in CSV format.
- To display the sales analysis using a Flask web application.

Description of Algorithm

1. Start the application.
2. Read the product details from the **products.json** file.

3. Read the sales transactions from the **sales.csv** file.
4. Merge both datasets using the **Product ID**.
5. Calculate the total units sold for each mobile phone.
6. Calculate the total revenue for each product.
7. Identify the best-selling mobile phone based on units sold.
8. Generate the **sales_report.csv** file.
9. Display the sales summary and product details in the MobileMart Analytics dashboard.
10. Stop the application.

Input

The application uses the following input files:

products.json

Contains:

- Product ID
- Brand
- Model
- Price

sales.csv

Contains:

- Sale ID
- Product ID
- Quantity Sold
- Date

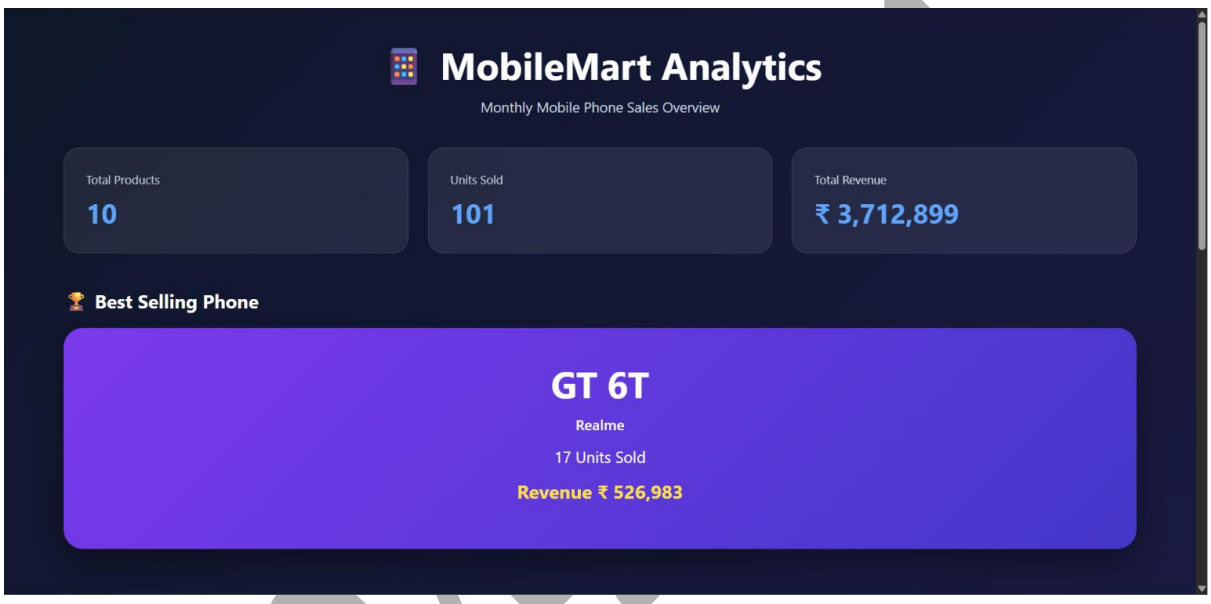
Output

The application successfully:

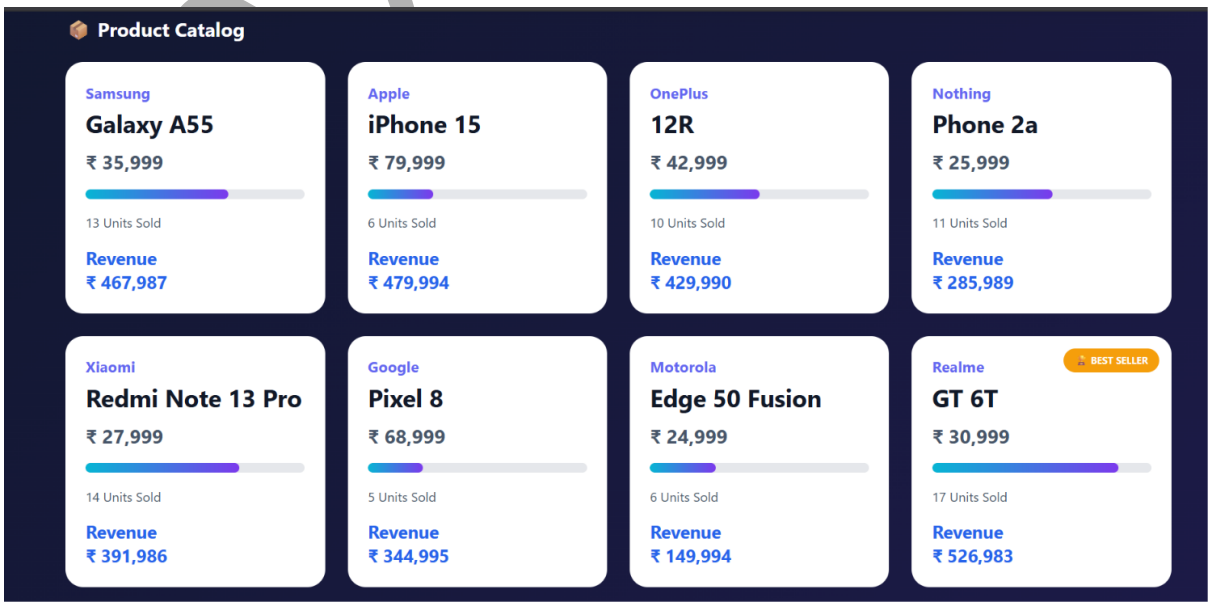
- Reads product and sales datasets.

- Combines product and sales information.
- Calculates total units sold and revenue for each mobile phone.
- Identifies the best-selling product.
- Generates the **sales_report.csv** file.
- Displays the MobileMart Analytics dashboard with product cards, sales summary, and best-selling phone.

Include the following screenshots in the report



- **Figure 1:** MobileMart Analytics Dashboard



- **Figure 2:** Product Catalog Section

products.json | sales.csv | data_loader.py | sales_pr

data > products.json > ...

```
[
  {
    "product_id": "P101",
    "brand": "Samsung",
    "model": "Galaxy A55",
    "price": 35999
  },
  {
    "product_id": "P102",
    "brand": "Apple",
    "model": "iPhone 15",
    "price": 79999
```

- **Figure 3:** products.json Dataset

products.json | sales.csv | data_loader.py

data > sales.csv

```
sale_id,product_id,quantity,date
S001,P101,4,2026-07-01
S002,P102,2,2026-07-01
S003,P103,5,2026-07-02
S004,P104,6,2026-07-02
S005,P105,3,2026-07-03
S006,P101,5,2026-07-03
S007,P106,2,2026-07-04
```

- **Figure 4:** sales.csv Dataset

sales.csv | sales_report.csv | data_loader.py | sales_pr

data > sales_report.csv

```
product_id,brand,model,price,Units Sold,Revenue
P101,Samsung,Galaxy A55,35999,13,467987
P102,Apple,iPhone 15,79999,6,479994
P103,OnePlus,12R,42999,10,429990
P104,Nothing,Phone 2a,25999,11,285989
P105,Xiaomi,Redmi Note 13 Pro,27999,14,391986
P106,Google,Pixel 8,68999,5,344995
P107,Motorola,Edge 50 Fusion,24999,6,149994
P108,Realme,GT 6T,30999,17,526983
P109,Vivo,V30,33999,8,271992
P110,Oppo,Reno 11,32999,11,362989
```

- **Figure 5:** Generated sales_report.csv

app.py
```python
def dashboard():
    products = load_products()
    sales = load_sales()

    # Process sales
    report = process_sales(products, sales)

    # Generate CSV report
    generate_report(report)

    # Dashboard statistics
    total_products = len(report)
    total_units = report["Units Sold"].sum()
    total_revenue = report["Revenue"].sum()
```

src > sales_processor.py > process_sales
```python
def process_sales(products_df, sales_df):
    # Merge products and sales
    merged = sales_df.merge(products_df, on="product_id", how="left")

    # Calculate revenue
    merged["Revenue"] = merged["quantity"] * merged["price"]

    # Group by product
    report = (
        merged.groupby(
            ["product_id", "brand", "model", "price"],
            as_index=False
        )
```

src > report_generator.py > generate_report
```python
def generate_report(report_df):
    report_df.to_csv(
        "data/sales_report.csv",
        index=False
    )
```

- **Figure 6:** Source Code (app.py, sales_processor.py, report_generator.py)

Conclusion

The **MobileMart Analytics** application successfully integrates product information from a JSON file with sales transaction data from a CSV file to analyze mobile phone sales performance. It calculates the total units sold and revenue for each product, identifies the best-selling mobile phone, and generates a consolidated sales report. The Flask-based web interface presents the information in a modern and user-friendly format,

demonstrating practical implementation of data integration, data processing, report generation, and web application development using **Python, Flask, Pandas, HTML, CSS, JSON, and CSV**.
